Software Test Plan (STP) - Complete Me

Or Reshef S 324064849 Gal Azoulay 323859967

1. Introduction

CompleteMe is a web-based puzzle game designed primarily for children. Users are shown an image with one or more missing sections and must select the piece they believe fits from a set of six options. What makes the platform distinctive is that correctness is determined not by coordinate matching but by a computer vision pipeline that analyzes boundary colors, texture and deep learning features extracted via ResNet50.

The system consists of four main components: a React frontend, a Flask REST API backend, a PostgreSQL database and the multi algorithm CV and DL validation engine. This document lays out the testing plan for the project what we intend to test, how we plan to go about it and what we are explicitly leaving out of scope for this cycle. The goal is to ensure that every meaningful part of the system from user registration through to puzzle validation and score recording works the way it should.

2. Test Items

2.1 Backend Modules

backend/app.py — Main Flask Application

This is the central entry point of the server. It sets up all the Flask extensions (SQLAlchemy, JWT, Bcrypt, CORS, Rate Limiter) and registers the API routes for puzzle creation, validation, hint generation, cleanup, health checks and statistics. We will verify that the application starts cleanly, that all routes are correctly registered and that extensions are properly initialized before any request is handled.

backend/routes/auth.py — Authentication Routes

This module handles everything related to user accounts: registration, login, logout, token refresh, profile retrieval, avatar updates, password changes, game history access and account deletion.

backend/models/cv_validator.py — CV Validator

This component decides whether a user's selected piece is correct. It does so using a relative ranking approach: the user's piece is scored alongside all six available options and it is considered correct only if it ranks first. We will test that the ranking logic is applied consistently and that edge cases such as very similar scores between two pieces are handled predictably.

backend/models/puzzle_generator.py — Puzzle Generator

This module is responsible for slicing the image into a puzzle. It supports five difficulty levels (2, 4, 8, 16, and 32 pieces) and between one and four missing regions. Its output is a modified puzzle image with black squares covering the missing areas, a set of six candidate pieces and the coordinates of each missing region.

backend/models/image_processor.py — Image Processor

Handles low-level image operations: loading, resizing, applying the black square mask and cropping individual pieces. We will verify that the MAX_IMAGE_DIMENSION constraint is respected and that output images are always in the correct format and dimensions.

backend/models/user.py / game_history.py — Database Models

SQLAlchemy ORM models for the two main database tables. Test items include password hashing and verification, the to_dict() serialization method, field constraints (unique username and email, avatar IDs limited to 1-5), and ensuring that deleting a user account also removes all associated history records through cascade delete.

backend/utils/boundary_matcher.py — Boundary Matcher

The primary fast-path validation algorithm. It extracts thin color strips from the four edges of the black square in the puzzle image and compares them against the corresponding edges of each candidate piece using color histogram analysis. We will verify that the strip extraction is geometrically correct and that the comparison scores are consistent and meaningful.

backend/utils/feature_extraction.py / color_analysis.py — CV Utilities

These utilities support the deeper side of the validation pipeline. The FeatureExtractor loads a pretrained ResNet50 and produces feature vectors from 224×224 inputs with ImageNet normalization. The ColorAnalyzer uses K-Means clustering to identify dominant colors and compares pieces via histogram matching. We will test that inputs are preprocessed correctly and that outputs fall within expected value ranges.

backend/utils/unsplash_api.py — Unsplash API Client

A wrapper around the Unsplash API that retrieves images based on a search query. It also includes a fallback mode that generates a synthetic image locally when no API key is configured which is particularly useful for offline and unit testing scenarios. We will verify that both the API path and the fallback path produce valid and usable output.

2.2 Frontend Modules

frontend/src/App.jsx is the top-level component of the application. It manages the overall routing and global state — including the current user, selected difficulty, region count, active game data, and score — and controls the transitions between the different screens as the user moves through the application.

frontend/src/services/api.js is the single communication layer between the frontend and the backend. It wraps all HTTP calls: puzzle creation, validation, hint requests, auth operations, history retrieval and profile updates into a clean service interface.

frontend/src/components contains all the visual components: WelcomeScreen, ImageSelector, GameBoard, ValidationModal, LoginScreen, SignupScreen, LoadingScreen, HistoryScreen, EditProfileScreen, UserBar, OptionsGrid, PuzzleImage. Each will be tested for correct rendering, interaction handling and appropriate state updates

2.3 Database

The PostgreSQL schema itself is a test item. This includes the structure of the users and game_history tables, their column constraints, the foreign key relationship between them, and the cascade delete behavior that removes all history records when a user account is deleted. In our project we do it at the application-level, so the route manually deletes history first, then deletes the user.

2.4 API Endpoints

All our API endpoints are in scope for testing:

- POST /api/puzzle/create : starts a new game session
- POST /api/puzzle/validate : checks the user's answer
- POST /api/puzzle/hint : returns a contextual hint
- POST /api/puzzle/cleanup : frees server-side game resources
- GET /api/health : basic server health check
- GET /api/config : returns difficulty definitions and settings
- All 9 /api/auth/ endpoints: register, login, logout, refresh, profile, update-avatar, change-password, history, delete-account

3. Features to be Tested

3.1 User Authentication and Account Management

Registration should work correctly when all required fields are valid: a username of at least 3 characters, a password of at least 6 characters, a properly formatted email and an avatar selection from the five available. The system should reject attempts to register with a username or email that already exists in the database and should set JWT cookies upon successful account creation.

Login should authenticate users with correct credentials and redirect them to the Welcome screen. Wrong passwords or unrecognized emails should return a 401 error. After 5 failed attempts within a 15-minute window, rate limiting should prevent further tries. Guest mode should be accessible without credentials and once in guest mode, score tracking and history saving should be disabled.

Logout should clear JWT cookies and cause any subsequent attempt to reach an authenticated route to redirect back to the login screen.

Token refresh should issue a new access token when the current one expires, provided the refresh cookie is still valid. An invalid or expired refresh token should redirect the user to login.

Avatar updates should apply immediately after selecting a different avatar on the Edit Profile screen, the UserBar should reflect the change without requiring a page reload. Attempting to set an avatar ID outside the range of 1–5 should be rejected with a 400 error.

Password changes should succeed when the user provides their correct current password alongside a valid new one and fail with a 401 when the current password is wrong.

Account deletion requires password confirmation. If successful, the user record and all associated game history should be permanently removed (like cascade delete). JWT cookies should be cleared and the user redirected to the login screen. Entering the wrong password should block the deletion entirely.

3.2 Game Setup

Difficulty selection on the Welcome screen should correctly map each button (Beginner/ Easy/ Medium/ Hard/ Expert) to its corresponding piece count (2/ 4/ 8/ 16/ 32). The selected option should be visually highlighted and the Continue button should remain disabled until a selection is made.

Region count selection should allow choosing between 1 and 4 missing pieces, with the black square indicators updating accordingly. The range of available region counts should adjust based on the chosen difficulty to keep combinations sensible.

Image source selection covers three paths: keyword search (sends the user's text to the puzzle creation API), category selection and the Surprise Me mode which picks a random category. The Back button should return to the Welcome screen without losing the difficulty selection the user already made.

3.3 GameBoard

Drag-and-drop When a user drags a piece from the options pool and drops it onto a missing zone, the piece should be recorded at that zone. Hovering over a valid zone while dragging should show a visual highlight. Dropping a piece onto a zone that already has a piece should replace it, with the original piece returning to the pool as available. Clicking a placed piece should remove it from the zone and make it selectable again.

The Hint button should only be enabled when exactly one zone remains unfilled. When clicked, it calls the hint endpoint and displays a contextual description of the boundary

colors around that zone. The hint should help the user narrow down their choice without revealing the answer directly.

The Check Answer button should be disabled until all zones have a piece placed. Once clicked, it submits the current placements to the validation endpoint and increments the attempt counter.

3.4 Answer Validation

Validation Modal should clearly communicate whether the answer was correct or incorrect. It should display a confidence score with color to reflect how strong the result is: green for scores above 0.85, blue for 0.70–0.85, orange for 0.50–0.70 and red for anything below that.

Backend validation works by scoring all six candidate pieces using the validate_piece_placement() function, then checking whether the piece the user chose ranked first. When two scores fall within a margin of 0.02 of each other (the TIE_MARGIN), validate_comprehensive() is called as a tiebreaker.

Scoring should follow the formula: correct answers earn max(10, 100 − (attempts − 1) × 10) points, so a first correct attempt earns 100, the second earns 90 and so on with a minimum of 10 points. Incorrect answers earn nothing. For logged-in users the total score should be updated in the database at the end of each game.

3.5 History and Stats

The History screen should display completed games in reverse chronological order, with columns for date, difficulty, grid size, number of missing pieces and score earned. Summary statistics are at the top with total games, total score and average score. The History screen should be inaccessible for guest users.

The UserBar should always show the current user’s avatar and username or "Guest" when not logged in. Navigation buttons for New Game, History, Edit Profile and Logout should work correctly from any authenticated screen.

4. Features Not to be Tested

AI model accuracy as a metric - We are not running the CV models against a labeled dataset to measure their precision or recall. The scope here is whether the validation pipeline integrates correctly and produces reasonable outputs.

Cross-browser compatibility - Testing is limited to the latest version of Google Chrome. We won’t test Safari, Firefox and Edge for this cycle.

Mobile and touch interactions - The drag-and-drop mechanic on touchscreens and tablet viewports will not be tested. CompleteMe is treated as a desktop application for the purposes of this submission.

Performance and load testing - We are not simulating high-concurrency scenarios or running throughput benchmarks in this cycle. This is deferred to a dedicated performance testing phase.

5. Testing Strategy

Our approach combines four levels of testing, each targeting a different layer of the system.

Unit testing focuses on individual functions in isolation. Things like the score calculation formula, password hashing and verification, individual CV utility functions and the rendering behavior of individual React components. The goal here is to catch bugs at the lowest level, where they are easiest to diagnose and fix.

Integration testing verifies that the parts of the system work together correctly. A typical integration test might walk through the full flow of creating a puzzle, submitting an answer and checking that the result appears correctly in the game history table.

System testing simulates complete user sessions from start to finish. Launching the app, setting up a game, playing through it and seeing the score recorded.

Acceptance testing The goal is to confirm that the system behaves correctly from a real user's perspective, not just from a technical one so itis a manual walkthrough of the main user scenarios: a first-time user registering and playing their first game, a returning user logging in and reviewing their history, a guest user completing a puzzle without saving progress.

6. Test Environment

Backend: Python 3.10+, Flask 3.1.2, PostgreSQL 14+, PyTorch 2.9.1, OpenCV 4.12.0. Environment variables loaded via .env (Unsplash API key, database URI, JWT secret).

Frontend: Node.js 16+, React 18.2.0, tested on Google Chrome (latest).

Testing Tools: If we were to carry out all the tests described in this plan, we would use tools such as pytest and pytest-flask for backend unit and integration tests, Jest and React Testing Library for frontend component tests and Postman for manual API endpoint verification. For inspecting the database state, we would use pgAdmin.

Test Data: If we were to run the full test suite, we would work with a seeded test database containing pre-created user accounts and game history records. Local sample images would be used in unit tests to avoid depending on the live Unsplash API and mocked Unsplash responses would be prepared for offline testing scenarios where network access is unavailable or unreliable.